OT

26.03.20 (금) 오후 6시 ~ 8시

정보과학관 21403

목차

- 스터디 소개
- PS를 위한 C++
- 재귀
- 비트마스킹 (시간 여유가 있다면 다루겠으나, 3차시에 다룰 예정)
- 백트래킹

스터디 소개

스터디 소개 | 01 OT

- 스터디 내용
  - PS에서 사용하는 기초적인 개념과 실제 문제에 적용하는 방법
- 권장 대상
  - **Silver 5 ~ Gold 3 (solved.ac 클래스 2 ~ 4) 정도에서 다루는 알고리즘을 공부하고 싶은 분들**
- 선수 조건
  - C언어 문법: 입출력, 조건문, 반복문, 배열, 함수

- 무엇이든 **피드백**을 주시면 개선해 나가겠습니다.
- 강의 중간에 **이해가 되지 않는 부분**이나 **설명이 부족한 부분**이 있으면 편하게 말해주세요.
  - ~~PS를 쉰지 오래되어서 잘 대답 못할 수 있는데 1차적으로는 제미나이를 적극적으로 활용해주세요~~

스터디 소개 | 01 OT

- 스터디 시작 전에 출결 체크를 확인할 예정입니다.
  - 다른 조건들도 있지만, 2학기에 SCCC 잔류하려면 참석률 50% 이상(4회 이상 출석)이면 됩니다.
- 강의 자료는 스터디 시작 전 톡방에 올리겠습니다.
  - 예습이 필요없게끔 제가 최대한 노력하겠습니다.
- 스터디하는 모습을 촬영(영상 X)해야하니 양해부탁드립니다.
  - 쉬는 시간에 뒷모습만 나오게 찍을 예정이니 너무 걱정하지 말아주세요.
- 몇몇 차시는 예정된 시간보다 이르게 끝날 수 있습니다. 그때 이전 차시에 다루지 못한 내용을 다룰 예정입니다.
- **강의 내용과 일정이 수정되어서 다시 안내해드리겠습니다.**
  - 시험 등의 이유로 결석할 경우, 따로 연락 남겨주세요.
- 강의 장소는 고정이 아니기에 매주 공지해드릴 예정입니다.
- 강의 중간에 쉬는 시간 겸 문제를 어떻게 풀면 될지 고민하는 시간을 드립니다.
  - 당연하지만, 쉬는 시간이 아니여도 편하게 화장실 다녀오셔도 됩니다.
  - 문제 풀 시간(생각 -> 구현 -> AC)은 드리진 않습니다.

스터디 소개 | 01 OT

1차시	26.03.20(금)	OT : PS를 위한 C++, 재귀, 백트래킹, 비트마스킹	solved.ac 활용법과 C에서 C++로의 효율적인 PS 전환 가이드를 학습합니다. 이후 실전 문제 해결의 필수 기반인 재귀와 백트래킹, 성능 최적화를 위한 비트마스킹 기법을 학습합니다.
2차시	26.03.27(금)	알고리즘 기초 : 시간 복잡도, 정렬, 투 포인터, 매개변수 탐색, 이분 탐색, 분할 정복	짜야하는 알고리즘의 성능을 추정하기 위해 시간복잡도에 대해 학습합니다. 이어 정렬 문제를 다룬 후, 효율적인 탐색을 위한 투 포인터와 이분 탐색 기법을 익힙니다. 그다음 분할 정복을 통해 1차시에서 배운 재귀 개념을 복습합니다. 이후 시간이 허락한다면 매개변수 탐색까지 다룰 예정입니다.
3차시	26.04.03(금)	정수론 기초: GCD와 LCD, 소인수 분해, 에라토스테네스의 체, 유클리드 호제법, 거듭제곱	solved.ac 실버에서 골드 난이도 수학 문제 풀이에서 자주 등장하는 정수론의 주요 아이디어를 다룹니다. 최대공약수(GCD)와 최소공배수(LCM)의 기본 원리를 익히고 나머지 연산의 특징을 이해합니다. 이를 바탕으로 두 수의 최대공약수를 빠르게 찾아내는 유클리드 호제법 알고리즘을 학습합니다. 마지막으로는 매우 큰 지수 계산을 효율적으로 처리하는 거듭제곱 기법을 탐구하며, 지난 2차시에 배웠던 분할 정복 기법을 복습하는 시간을 갖습니다.
4차시	26.04.10(금)	동적 계획법	동적 계획법(이하 'DP')은 큰 문제를 작은 부분 문제로 나누어 해결하되, 한 번 계산한 답을 저장해 두었다가 재활용하는 효율적인 알고리즘 설계 기법입니다. 이번 차시에서는 DP 문제를 푸는 방법을 간단히 안내하고, 배낭 문제나 타일링과 같은 10가지 문제를 통해 dp 배열을 정의하고 점화 관계를 찾아내는 연습을 기계적으로 적용하는 연습을 합니다.
휴강	5주차 ~ 8주차	중간고사 휴강	

스터디 소개 | 01 OT

5차시	26.05.01(금)	그리디, 자료구조 (1) : 스택, 큐, 덱	그리디는 국소 최적이 전역 최적에 도달하기를 기대하는 알고리즘 설계 방식입니다. 그리디 기법과 저번 차시에 배운 DP 기법을 비교하며, 그리디와 DP가 어떤 차이가 있는지 알아 봅니다. 이후에는 스택, 큐, 덱과 같은 자료구조의 사용법에 대해서 학습합니다. 각각의 자료구조가 제공하는 연산 위주로 학습합니다. 각각의 연산에 대한 시간복잡도와 자료구조의 연산 특징을 통해 알고리즘 문제를 풀 때 어떤 자료구조를 선택해야하는지 감을 잡아보는 연습을 합니다.
6차시	26.05.8(금)	자료구조 (2) : 그래프, 깊이 우선 탐색, 너비 우선 탐색, 이진 트리, 위상 정렬	수학에서의 그래프가 아닌 이산 구조로서의 그래프를 학습합니다. 그래프에 대한 기초적인 용어 정의부터 시작하여, 모든 정점들을 방문하는 여러가지 방법에 대해 알아보고, 그래프의 특수한 형태인 트리에 대해 학습합니다. 이진 트리에 관한 내용을 간단하게 살펴본 후, 정점 간의 선후 관계를 고려하여 순서를 정하는 위상 정렬을 학습합니다.
7차시	26.05.15(금)	자료구조 (3) : 이진 탐색 트리, 우선순위 큐(힙), 서로소 집합, 최소 스패닝 트리	골드 상위권에 위치하는 알고리즘을 학습합니다. 자료구조의 개념, STL 사용법(또는 구현 방법), 문제 적용 방법을 학습합니다.
8차시	26.05.22(금)	최단 경로 : 다익스트라, 플로이드 워셜	최단 경로 문제에 대해 학습합니다. 하나의 정점에서 모든 다른 정점까지의 최단 경로를 계산하는 다익스트라 알고리즘, 모든 정점 쌍에 대해 최단 경로를 구하는 플로이드 워셜 알고리즘에 대해 학습합니다.
휴강	13주차 ~ 15주차	기말고사 휴강	종강

스터디 소개 | 02 solved.ac 란?

- 백준 문제를 **난이도별로 정리**하고, **티어 시스템**을 적용한 랭킹 사이트

레벨	미해결	해결	전체	진행도
모든 문제	모두 보기	모두 보기	모두 보기	
Unrated	5,915	3	5,918	
Bronze V	72	92	164	
Bronze IV	270	59	329	
Bronze III	683	74	757	
Bronze II	897	72	969	
Bronze I	764	51	815	
Silver V	844	60	904	
Silver IV	855	65	920	
Silver III	969	70	1,039	
Silver II	909	84	993	
Silver I	977	61	1,038	
Gold V	1,152	74	1,226	
Gold IV	1,583	84	1,667	
Gold III	1,455	52	1,507	
Gold II	1,330	42	1,372	
Gold I	1,101	32	1,133	
Platinum V	1,314	28	1,342	
Platinum IV	1,284	17	1,301	
Platinum III	1,377	8	1,385	

스터디 소개 | 02 solved.ac 란?

- 문제에 **태그**가 붙어 있어서, 특정 유형의 문제를 연습 가능

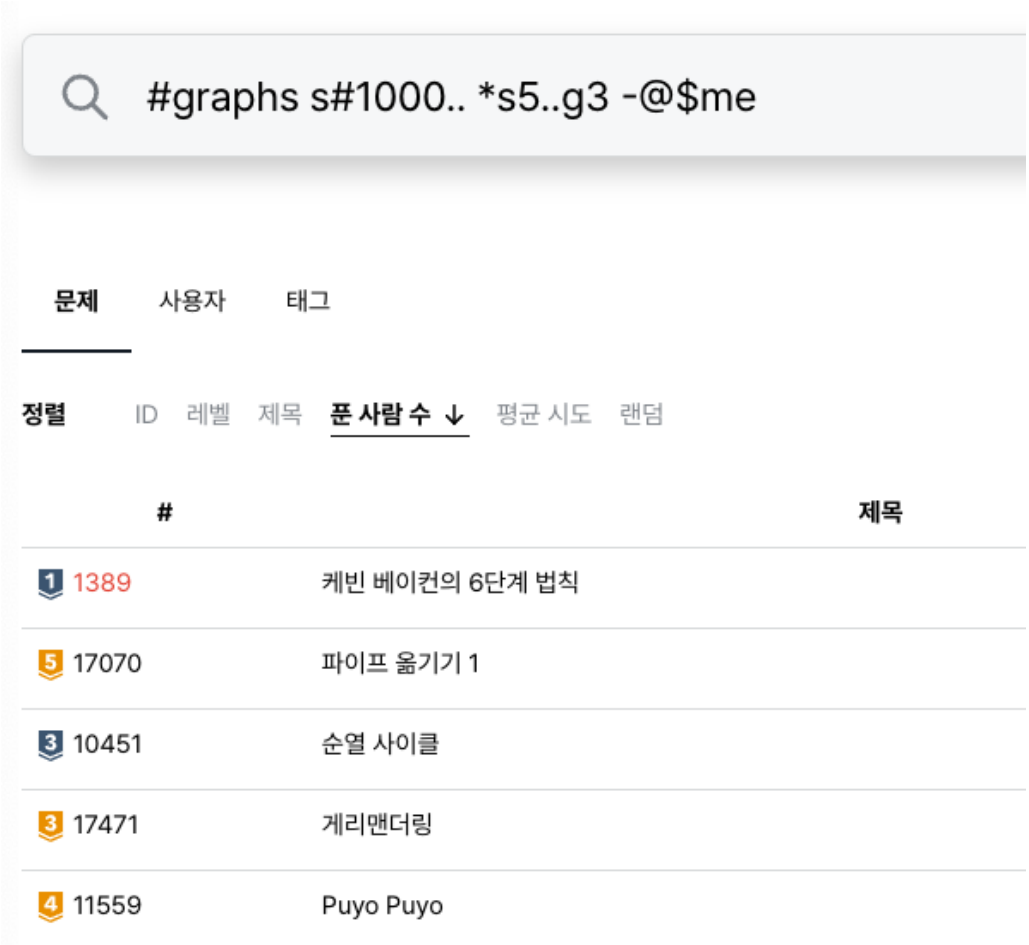
#그래프 탐색 graph_traversal

정렬 ID 레벨 제목 푼 사람 수 ↓ 평균 시도 랜덤

#	제목	푼 사람 수	평균 시도
2 1260	DFS와 BFS STANDARD	75,453	2.60
1 2178	미로 탐색	67,199	2.22
3 2606	바이러스 STANDARD	64,429	2.17
1 2667	단지번호붙이기	60,168	2.31
2 1012	유기농 배추 STANDARD	59,166	2.56
5 7576	토마토 STANDARD	54,048	2.66

스터디 소개 | 02 solved.ac 란?

- **검색**을 통해 풀고 싶은 문제를 찾을 수 있음.



#graphs : 그래프 태그가 붙여진 문제

s#1000.. : 1000명 이상 푼 문제

*s5..g3 : 실버 5와 골드 3 사이의 문제

-@$me : 내가 푼 문제를 제외하고

https://solved.ac/search 참고

스터디 소개 | 02 solved.ac 란?

- CLASS : PS에서 마주치게 되는 주제와 트릭을 단계별로 모아둔 학습 로드맵
  - 클래스를 달성하면 레이팅을 '꽤 많이' 올릴 수 있음.

스터디 소개 | 03 알고리즘 문제 푸는 과정

1. 문제를 정확히 이해하기.
   - 입력과 출력 형식 확인 : 어떤 데이터가 주어지고, 어떤 형식으로 결과를 출력해야 하는지 확인.
   - 제약 사항 확인 : 데이터의 크기, 특정한 조건(메모리 제한, 시간 제한) 등을 확인.
   - 예제 분석 : 주어진 예제를 직접 손으로 풀어보면서 문제의 패턴을 파악.
2. 풀이를 생각하기
   - 시간 복잡도와 공간 복잡도를 계산해서 대략 어느 정도 성능을 가진 알고리즘을 생각해내야 하는지 판단하기.
   - 컴퓨터는 간단한 동작을 1초에 1억(1e8) ~ 10억(1e9)번 할 수 있음.
     - n = 5000이면 O(N^2) 정도의 알고리즘을 작성하면 되겠구나.
     - n = 200,000이면 O(n log n) 정도의 알고리즘을 작성하면 되겠구나. // 알고리즘에서 log의 밑이 명시되지 않았다면 $\log_2 N$이라고 생각하면 됩니다!
   - 수 하나는 4B ~ 8B이다
     - int a[10’000’000]은 40,000,000B = 40MB
     - int a[5’000][5’000]은 100,000,000B = 100MB

스터디 소개 | 03 알고리즘 문제 푸는 과정

3. 코드로 옮기기
4. 온라인 저지에 제출하기
5. 왜 틀렸는지 고민하기

맞왜틀 팁 : 엣지 케이스(가장 작은 케이스, 경계에 있는 케이스, 정말 큰 케이스)를 직접 테스트해보면 좋음.

(반복)

(반복)

(반복)

※ 안 풀린다고, 2~3시간 이상 너무 오래 고민하지 말기.

※ 다른 사람의 코드를 참고해서 공부하고 넘어가거나, 다음을 약속하기.

PS를 위한 C++

PS를 위한 C++ | 01 개요

- C로도 PS(Problem Solving : 문제 해결)를 할 수 있지만, C++를 추천!
  - C는 표준 라이브러리가 부족하기에 풍부한 C++을 쓰는게 이롭다.

    우선순위 큐를 구현하려면 힙 자료구조를 '직접' 만들어야 한다.
  - 백준 1753번 문제는 다익스트라 알고리즘을 사용하는 대표적인 문제이다.

    단적인 예시지만, C로 풀려면 C++로 풀 때에 비해 6배만큼 타이핑해야한다.

4 1753	맞았습니다!!	11044 KB	152 ms	C99	6621 B
4 1753	맞았습니다!!	9184 KB	88 ms	C++17	1097 B

PS를 위한 C++ | 02 입출력 (I/O) 방식

- C에서는 scanf/printf 를 사용하지만, C++에서는 cin/cout을 사용한다.
  - bits/stdc++.h 는 C++에서 PS할 때 자주 사용되는 모든 표준 라이브러리를 포함하는 헤더 파일이다.
  - C++에서는 표준 라이브러리(STL)를 사용할 때, std::cout 처럼 네임스페이스를 앞에 붙여줘야 한다.
  - using namespace std; 는 "std::"를 생략할 수 있도록 해주는 선언문이다.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
    ios::sync_with_stdio(false); // C와 C++의 입출력 동기화를 끊어 속도 향상
    cin.tie(nullptr); // 입력과 출력을 묶지 않아 속도 향상

    int a, b;
    cin >> a >> b; // scanf("%d %d", &a, &b); 와 같은 역할
    cout << a + b << "\n";// printf("%d\n", a + b); 와 같은 역할

    return 0;
}
```

PS를 위한 C++ | 03 C++ STL

- C++ STL(Standard Template Library)은 자료구조와 알고리즘을 쉽게 사용할 수 있도록 제공하는 표준 라이브러리이다.
  - 데이터를 저장하고 관리하는 자료구조를 제공한다. **컨테이너**라고 불린다.
    - vector, deque, list, array / set, map, unordered_set, unordered_map / stack, queue, priority_queue
  - 컨테이너에서 데이터를 조작하는 다양한 **알고리즘**을 제공한다.
    - sort, binary_search, max_element, min_element, next_permutation 등이 있다.
  - 컨테이너의 요소를 순회할 수 있는 객체를 제공한다. **이터레이터**라고 불린다.
    - vector<int>::iterator it 을 예시로 들 수 있다.
    - it은 vector<int>을 순회할 때 사용할 수 있는 반복자(이터레이터) 객체이다.

PS를 위한 C++ | 03 C++ STL (1) : vector

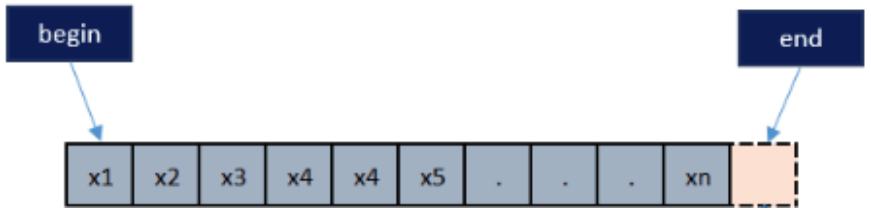
- vector는 동적 배열 기능(크기가 자동으로 조절되는 기능)을 제공하는 STL 컨테이너이다.
- 사용법

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main(){
    vector<int> v1; // 빈 벡터
    vector<int> v2(5); // 크기 5, 모든 요소는 0으로 초기화
    vector<int> v3(5, 10); // 크기 5, 모든 요소는 10으로 초기화
    vector<int> v4 = {1, 2, 3, 4, 5}; // 리스트 초기화
    vector<vector<int>> v5(5, vector<int>(6, 1)); // 5 x 6 크기, 모든 요소는 1로 초기화

    vector<int> v;
    v.push_back(1); // {1}
    v.push_back(2); // {1, 2}
    v.pop_back(); // {1} (마지막 요소 제거)

    cout << v.size(); // 1 (현재 크기)
    cout << v.capacity(); // 동적 할당된 크기

    cout << v[0]; // 일반 배열처럼 사용 가능 (O(1))
```

PS를 위한 C++ | 03 C++ STL (1) : vector

- vector는 동적 배열 기능(크기가 자동으로 조절되는 기능)을 제공하는 STL 컨테이너이다.
- 사용법

```
22      // 반복자 사용 (C++ STL에서 컨테이너의 원소를 순회하는 데 사용하는 객체. 배열에서 포인터처럼 동작하는 개념.)
23      v = {1, 2, 3, 4, 5};
24      for(vector<int>::iterator it = v.begin(); it != v.end(); it++){
25          cout << *it << " ";
26      } // 1 2 3 4 5
27
28      // 범위 기반 for문 (C++11 이상)
29      for(int x : v){
30          cout << x << " ";
31      }
32  }
```



PS를 위한 C++ | 03 C++ STL (2) : sort

- sort 함수는 배열이나 컨테이너(vector 등)의 원소를 정렬해준다.
  - 기본 동작은 오름차순 정렬을 한다.
  - greater<int>()를 사용해서 내림차순 정렬을 할 수 있다.
- 사용법

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
    vector<int> v = {3, 1, 5, 4, 2};

    sort(v.begin(), v.end()); // 오름차순 정렬
    sort(v.begin(), v.end(), greater<int>()); // 내림차순 정렬

}
```

PS를 위한 C++ | 03 C++ STL (2) : sort

- 사용자 정의 정렬 (람다 표현식을 활용한 예제)
  - 람다 표현식은 이름 없는 함수(익명 함수)를 만드는 문법으로 짧고 간결하게 함수 기능을 구현할 수 있다.

    [캡쳐](매개변수){함수 본문} 형태인데, 저렇게 쓸 수 있구나 정도로 형태만 간단히 기억하면 된다.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
    vector<int> v = {3, 1, 5, 4, 2};

    sort(v.begin(), v.end(), [](int a, int b){
        return a > b; // 내림차순 정렬
    });
    return 0;
}
```

PS를 위한 C++ | 03 C++ STL (3) : binary_search

- binary_search는 이분 탐색 기능을 제공하는 함수이다.
  - 찾는 값의 존재 유무를 리턴한다. true / false만 반환한다.
  - 값의 위치를 찾으려면 lower_bound를 사용하면 된다. 다음 차시에 다룰 예정이다.
- 사용법

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> arr = {1, 3, 5, 7, 9};
    cout << binary_search(arr.begin(), arr.end(), 3); // 1 (true)
}
```

PS를 위한 C++ | 03 C++ STL (4) : priority_queue

- C++에서는 힙을 기반으로 구현 된 priority_queue가 기본 제공된다.
  - C에서는 힙을 직접 구현해야 한다.
  - 힙은 최댓값 및 최솟값을 찾아내는 연산을 빠르게 하기 위해 고안된 자료구조이다.
- 사용법

```cpp
#include <iostream>
#include <queue>
using namespace std;

int main() {
    priority_queue<int> pq;  // 기본적으로 최대 힙

    pq.push(3);
    pq.push(1);
    pq.push(4);

    cout << pq.top() << "\n"; // 4
    pq.pop();                 // 가장 큰 값 제거

    return 0;
}
```

PS를 위한 C++ | 03 C++ STL (5) : 순열

- next_permutation()는 사전순으로 다음 순열을 생성하는 함수이다.
- prev_permutation()는 사전순으로 이전 순열을 생성하는 함수이다.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
    ios::sync_with_stdio(false);

    vector<int> a = {1, 2, 3, 4, 5};
    next_permutation(a.begin(), a.end());
    for(auto x : a) cout << x << " "; // 1 2 3 5 4
    prev_permutation(a.begin(), a.end());
    for(auto x : a) cout << x << " "; // 1 2 3 4 5
    return 0;
}
```

PS를 위한 C++ | 03 C++ STL (5) : 순열

- next_permutation()는 사전순으로 다음 순열을 생성하는 함수이다.
- 사용법 예) 모든 순열을 출력하기

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
    ios::sync_with_stdio(false);

    vector<int> a = {1, 2, 3};

    do{
        for(auto x : a) cout << x << " ";
        cout << "\n";
    }while(next_permutation(a.begin(), a.end()));

    return 0;
}
```

출력값

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

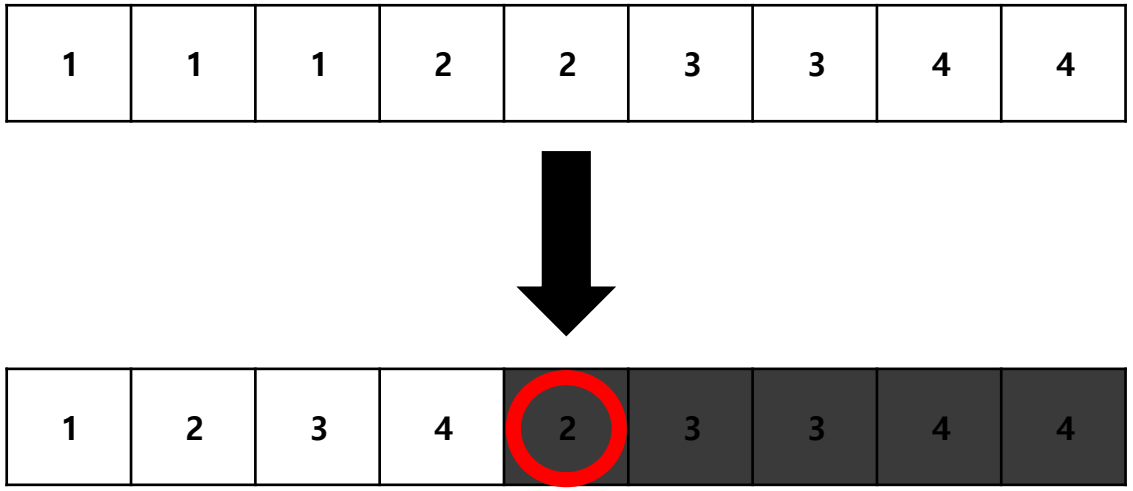
PS를 위한 C++ | 03 C++ STL (6) : 중복 제거

- 배열에서 중복된 값을 제거하기 위해 algorithm 헤더에 정의된 unique 함수와 vector 헤더에 정의된 erase 함수를 활용할 수 있다.
- unique 함수를 사용하기 전에 정렬을 먼저 수행해야한다.

PS를 위한 C++ | 03 C++ STL (6) : 중복 제거

- 사용법
  - unique 함수는 연속된 중복을 하나의 값으로 압축하고, 새로운 끝을 반환한다.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
    ios::sync_with_stdio(false);

    vector<int> v = {1, 1, 1, 2, 2, 3, 3, 4, 4};
    v.erase(unique(v.begin(), v.end()), v.end());
    for(auto x : v) cout << x << " "; // 1 2 3 4 출력함
    cout << "\n";
    return 0;
}
```



PS를 위한 C++ | 03 C++ STL (7) : 최대값, 최소값

- C 표준 라이브러리인 <stdlib.h>, <math.h> 에는 max 함수와 min 함수가 포함되어 있지 않다.
- C++ STL 에서는 max, min / max_element, min_element 함수를 제공한다.
- max, min 함수는 각각 최댓값과 최소값을 반환한다.
- max_element, min_element 함수는 각각 최댓값과 최솟값의 이터레이터를 반환한다.
- 사용법

```
int main() {
    vector<int> v = {3, 1, 4, 1, 5, 9, 2};

    auto maxIt = max_element(v.begin(), v.end()); // 최댓값 위치 찾기
    auto minIt = min_element(v.begin(), v.end()); // 최솟값 위치 찾기

    cout << "최댓값: " << *maxIt << "\n";  // *을 사용해 값 얻기
    cout << "최솟값: " << *minIt << "\n";

    return 0;
}
```

PS를 위한 C++ | 03 C++ STL (8) : pair, tuple

- C에서 구조체는 서로 다른 타입의 데이터를 하나로 묶어 사용하기 위해 사용된다.
- 이와 유사하게 C++에서는 pair, tuple를 제공한다.
  - pair 는 두 개의 값만 저장 가능하고 first, second로 접근할 수 있다.
  - tuple 은 세 개 이상의 값을 저장할 수 있고 get<index>(튜플 이름)으로 접근할 수 있다.
- 사용법

```cpp
#include <iostream>
#include <tuple>
using namespace std;

int main() {
    pair<int, string> p = {1, "apple"};
    cout << p.first << " " << p.second << "\n"; // 1 apple

    tuple<int, double, string> t = {1, 2.5, "hello"};
    cout << get<0>(t) << " " << get<1>(t) << " " << get<2>(t) << "\n"; // 1 2.5 hello
}
```

PS를 위한 C++ | 03 C++ STL (8) : pair, tuple

- 구조적 바인딩(Structured Binding, C++ 17 이상)을 통해 tuple, pair 등의 값을 쉽게 분해하여 변수에 할당할 수 있다.

```
    tuple<int, double, string> t = {1, 2.5, "hello"};
    auto [a, b, c] = t;
    cout << a << " " << b << " " << c << "\n"; // 1 2.5 hello
}
```

PS를 위한 C++ | 03 C++ STL (8) : pair, tuple

- PS에서 단독으로 사용하는 경우는 거의 없고 주로 컨테이너(vector, queue, set, map 등)와 같이 사용한다.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
    queue<pair<int, int>> q;

    q.push({1, 2});
    q.push({3, 4});
    while(!q.empty()){
        auto [a, b] = q.front();
        q.pop();
        cout << a << " " << b << "\n";
    }
}
```

PS를 위한 C++ | 03 STL C++ (9) : 스택, 큐, 덱

- 스택, 큐, 덱은 데이터를 저장하고 관리하는 방식을 정의하는 자료구조이다.
  - 스택 : 가장 나중에 삽입된 데이터가 가장 먼저 제거되는 자료구조 (LIFO : Last In First Out)
  - 큐 : 먼저 삽입된 데이터가 먼저 제거되는 자료구조 (FIFO : First In First Out)
  - 덱 : 스택과 큐의 특징을 모두 포함하는 자료구조
- C에서는 직접 구현해야하지만, C++에서는 이미 구현되어있다.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
    stack<tuple<string, string>> s;
    s.push(make_tuple("안녕", "하세요"));
    s.pop();
}
```

PS를 위한 C++ | 03 C++ STL (10) BST, Hash Table

- C++ STL에서는 이진 탐색 트리(BST, Red-Black Tree)와 해시 테이블(Hash Table)이 이미 구현되어 있다.
  - 이진 탐색 트리 : 정렬된 데이터를 빠르게 탐색할 수 있는 자료구조
  - 해시 테이블 : 키와 값의 쌍을 저장하는 자료구조로, 데이터를 빠르게 검색할 수 있도록 설계된 구조

➢ map: (key, value) 저장, 자동 정렬 (O(log N)) // BST 기반

➢ unordered_map: (key, value) 저장, 정렬 없음, 빠른 탐색 (O(1) 평균) // 해시 테이블 기반

➢ set: 중복 없는 원소 저장, 자동 정렬 (O(log N)) // BST 기반

➢ unordered_set: 중복 없는 원소 저장, 정렬 없음, 빠른 탐색 (O(1) 평균) // 해시 테이블 기반

PS를 위한 C++ | 04 결론

- **지금 단계에서는 전부 알고 계실 필요는 없습니다.**
  - 입출력
  - 벡터
  - 정렬

  위 3가지만 잘 기억해두시면, 4차시까지 문제없이 따라오실 수 있습니다.

  5차시부터는 본격적으로 자료구조를 학습하기 때문에 그 때 본격적으로 **사용법 위주**로 다시 소개해드릴 예정입니다.

재귀

재귀 | 01 재귀의 개념과 원리

- 재귀 함수란?

  자기 자신을 호출하는 함수!!!

  - 예) 피보나치 수열 : $F(n) = F(n-1) + F(n-2)$
  - 예) 하노이 탑 : $T(n) = 2T(n-1) + 1$
  - 예) 이진 탐색 : $T(n) = T(n/2) + O(1)$
  - 예) 팩토리얼 : $F(n) = n * F(n-1)$

```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```

자기 자신을 호출함

재귀 | 01 재귀의 개념과 원리

- **재귀 호출(Recursive Call)**과 **종료 조건(Base Case)**

  재귀 함수는 재귀 호출과 종료 조건이 있어야 한다.

  - 재귀 호출

    정의 : 함수 내부에서 자기 자신을 다시 호출하는 프로그래밍 기법.

    팁 : 주어진 문제를 작은 문제로 나누고, 이를 반복적으로 해결하는 과정에서 자연스럽게 등장한다.

  - 종료 조건

    정의 : 재귀 호출이 멈추는 조건.

    팁 : 보통 가장 작은 입력에 대해 직접 값을 반환한다.

```
int factorial(int n){
    if(n == 1) return 1;
    return n * factorial(n - 1);
}
```

종료 조건

재귀 호출

재귀 | 01 재귀의 개념과 원리

- 재귀와 반복문의 차이
  - 재귀를 알아야 하는 이유는..

    일반적인 for 문이나 while 문으로 해결할 수 없는 문제들이 존재. 재귀를 사용하면 간결하게 해결 가능.

    예) 트리 탐색, 그래프 탐색(DFS), 백트래킹 등은 재귀 없이는 비효율적이거나 구현이 복잡해질 수 있음

```
void preorder_traversal(TreeNode *root){
    if(!root) return;
    stack<TreeNode*> s;
    s.push(root);

    while(!s.empty()){
        TreeNode *node = s.top();
        s.pop();
        cout << node -> val << " ";

        if(node->right) s.push(node->right);
        if(node->left) s.push(node->left);
    }
}
```

```
void Preorder_traversal(TreeNode *root){
    if(!root) return;
    cout << root->val << " ";
    Preorder_traversal(root->left);
    Preorder_traversal(root->right);
}
```

재귀 | 01 재귀의 개념과 원리

- 재귀와 반복문의 차이
  - 재귀를 알아야 하는 이유는..

    분할 정복(Divide & Conquer) 알고리즘의 핵심이기 때문에 알아야 한다.

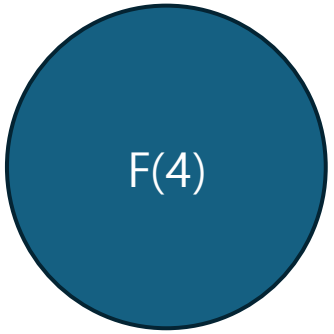
    예) 머지 소트, 퀵 소트, 이진 탐색, 평면상의 가장 가까운 두 점, 빠른 거듭제곱

```
// 구간 [left, right] 을 정렬하는 함수
void mergeSort(vector<int> &arr, int left, int right){
    if(left >= right) return; // Base Case
    int mid = (left + right) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}
```

재귀 | 01 재귀의 개념과 원리

```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```

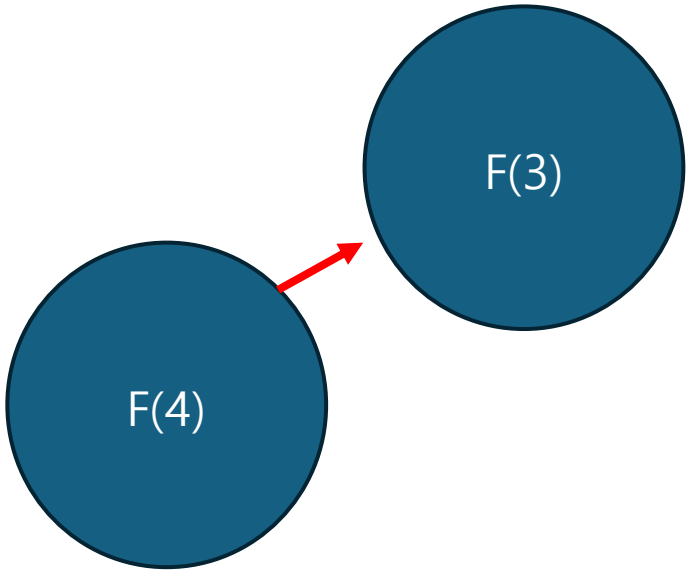
- 재귀 함수의 동작 방식 (예시, 피보나치 수열 : F(n) = F(n - 1) + F(n – 2))
- F(4)를 호출하면, 종료 조건을 만족하지 못하므로 F(n-1)을 호출하게 된다.



재귀 | 01 재귀의 개념과 원리

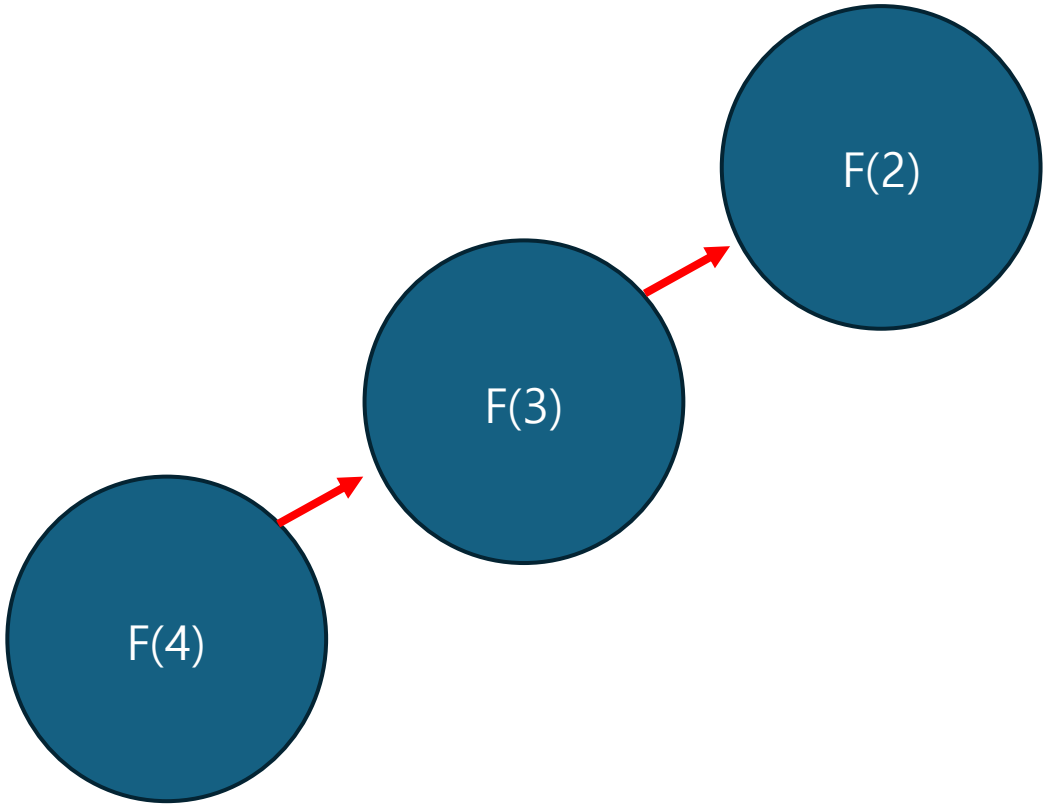
```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```

- 재귀 함수의 동작 방식 (예시, 피보나치 수열 : F(n) = F(n - 1) + F(n – 2))
- F(3) 역시 종료 조건을 만족하지 못하기에 F(2)를 호출한다.



재귀 | 01 재귀의 개념과 원리

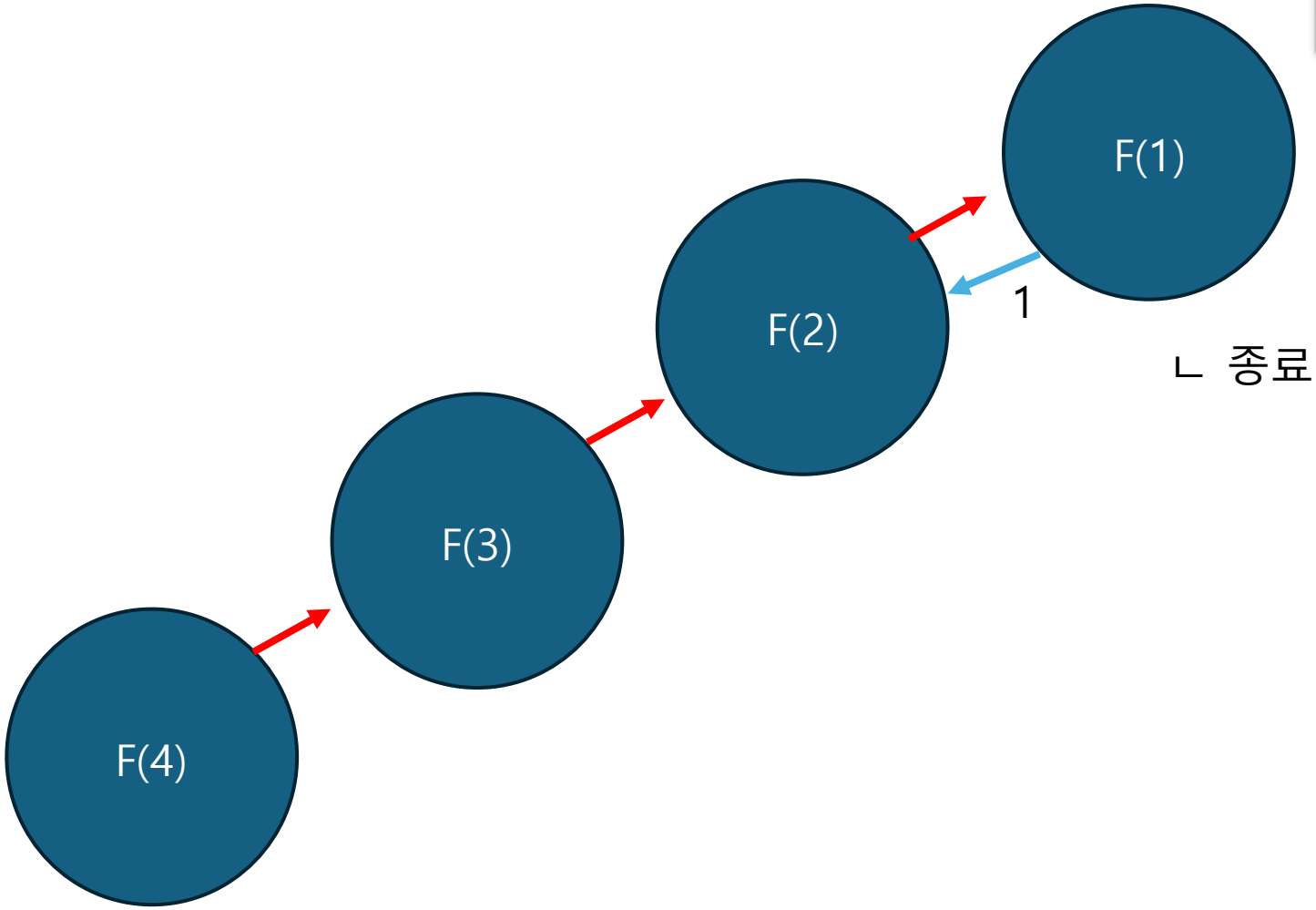
```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```



F(2)도 종료 조건 만족 못해서 F(1)를 호출한다.

재귀 | 01 재귀의 개념과 원리

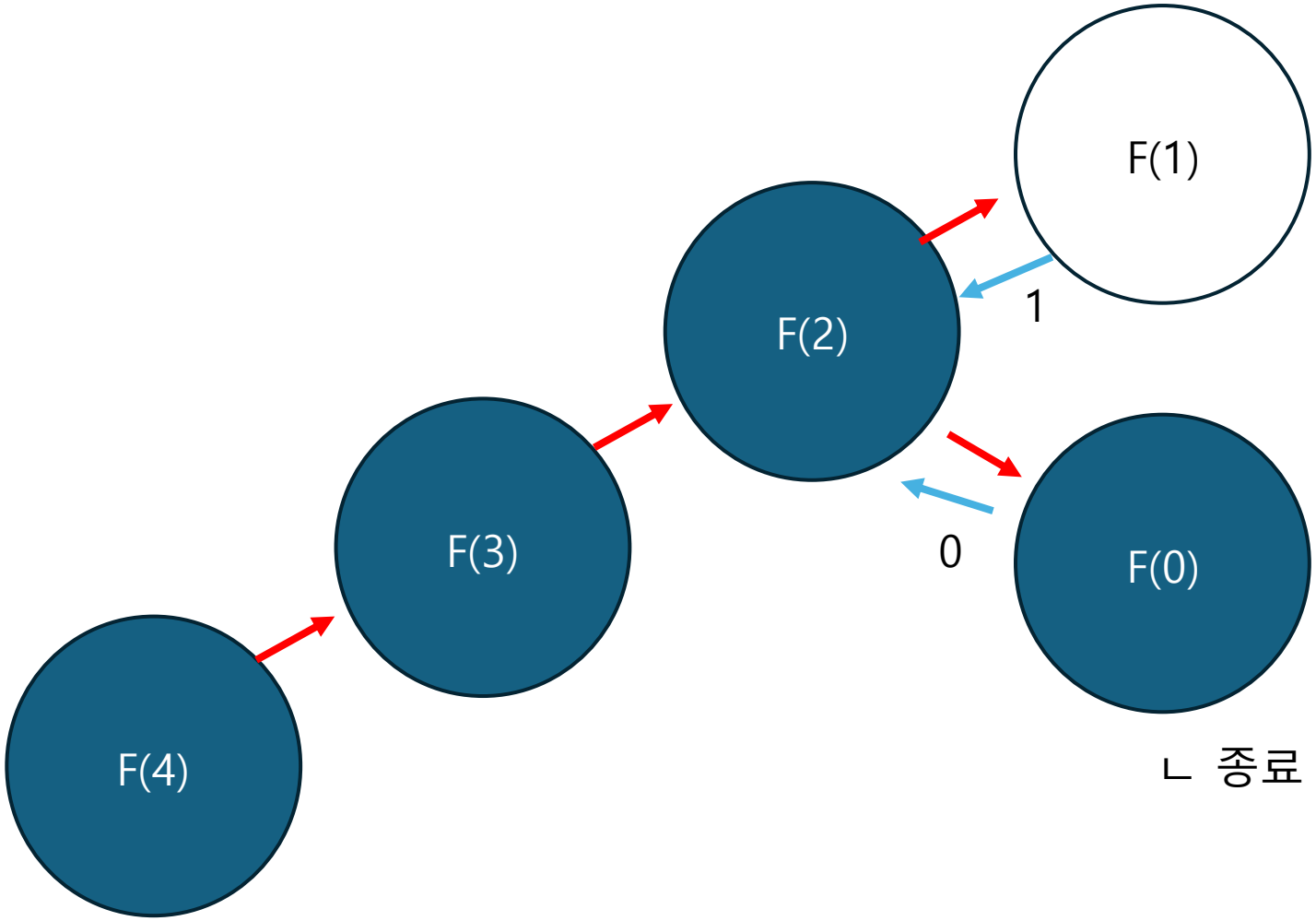
```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```



ㄴ 종료 조건에 만족하므로 1 반환한다.

재귀 | 01 재귀의 개념과 원리

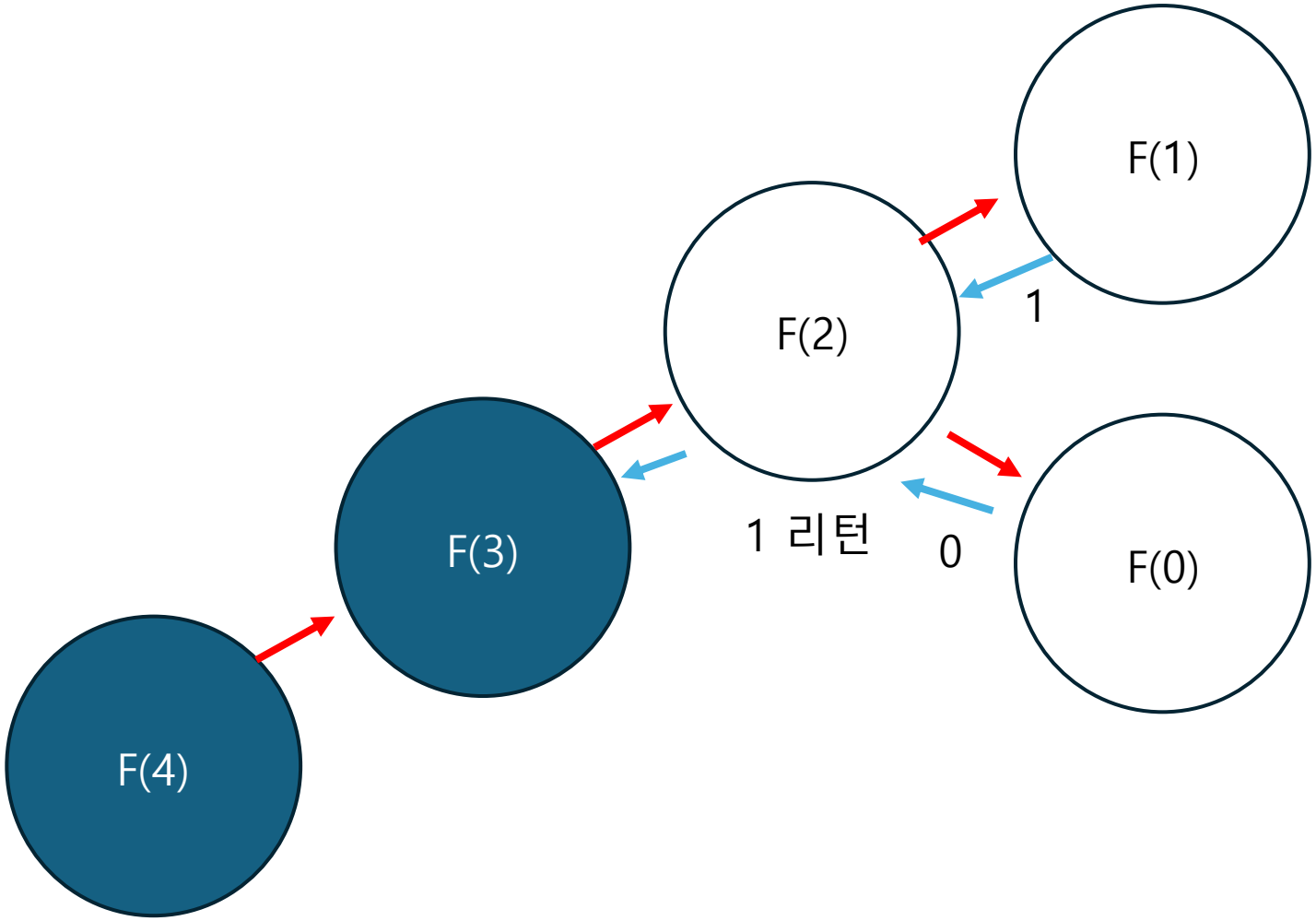
```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```



ㄴ 종료 조건에 만족하므로 0 반환한다.

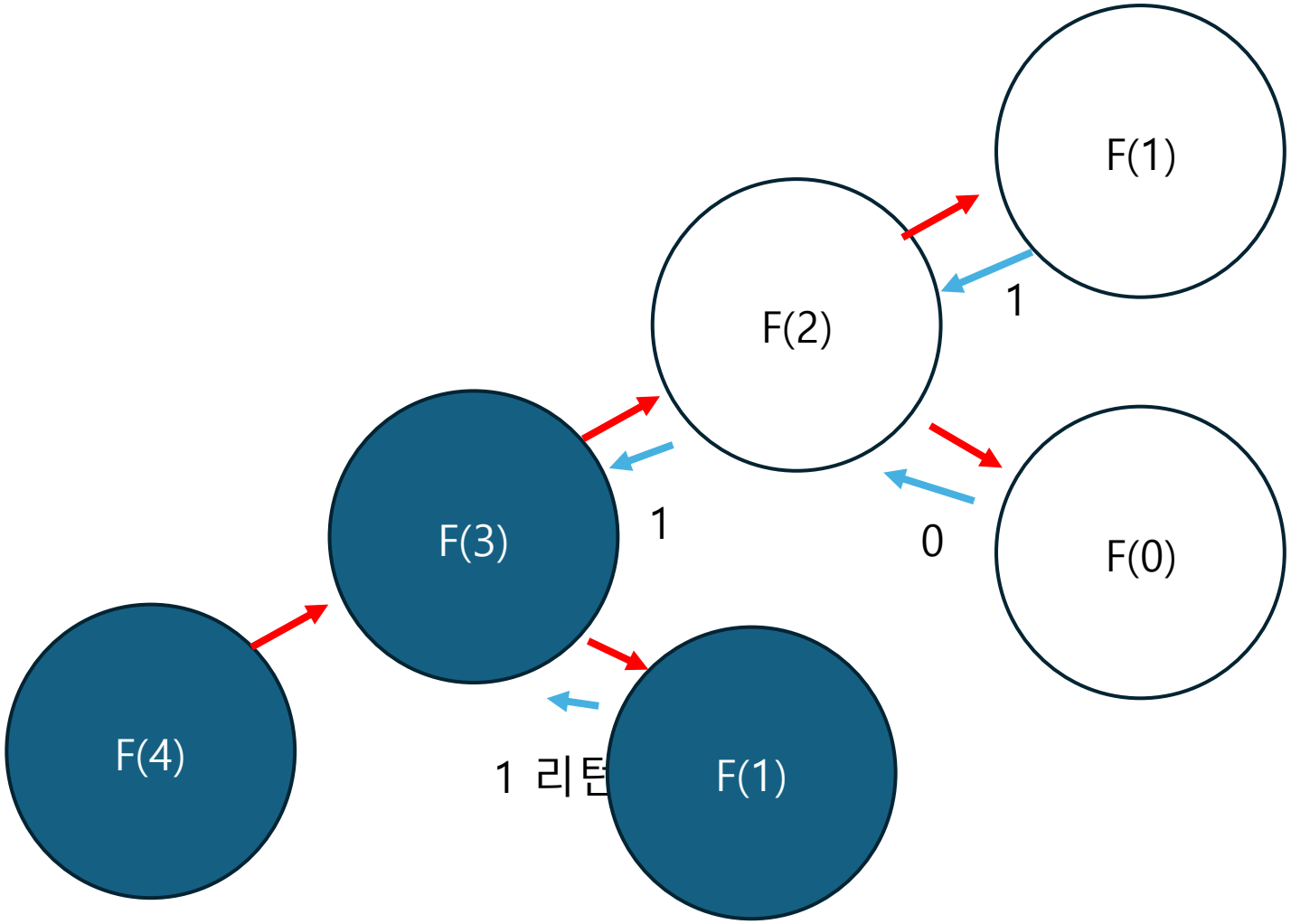
재귀 | 01 재귀의 개념과 원리

```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```



재귀 | 01 재귀의 개념과 원리

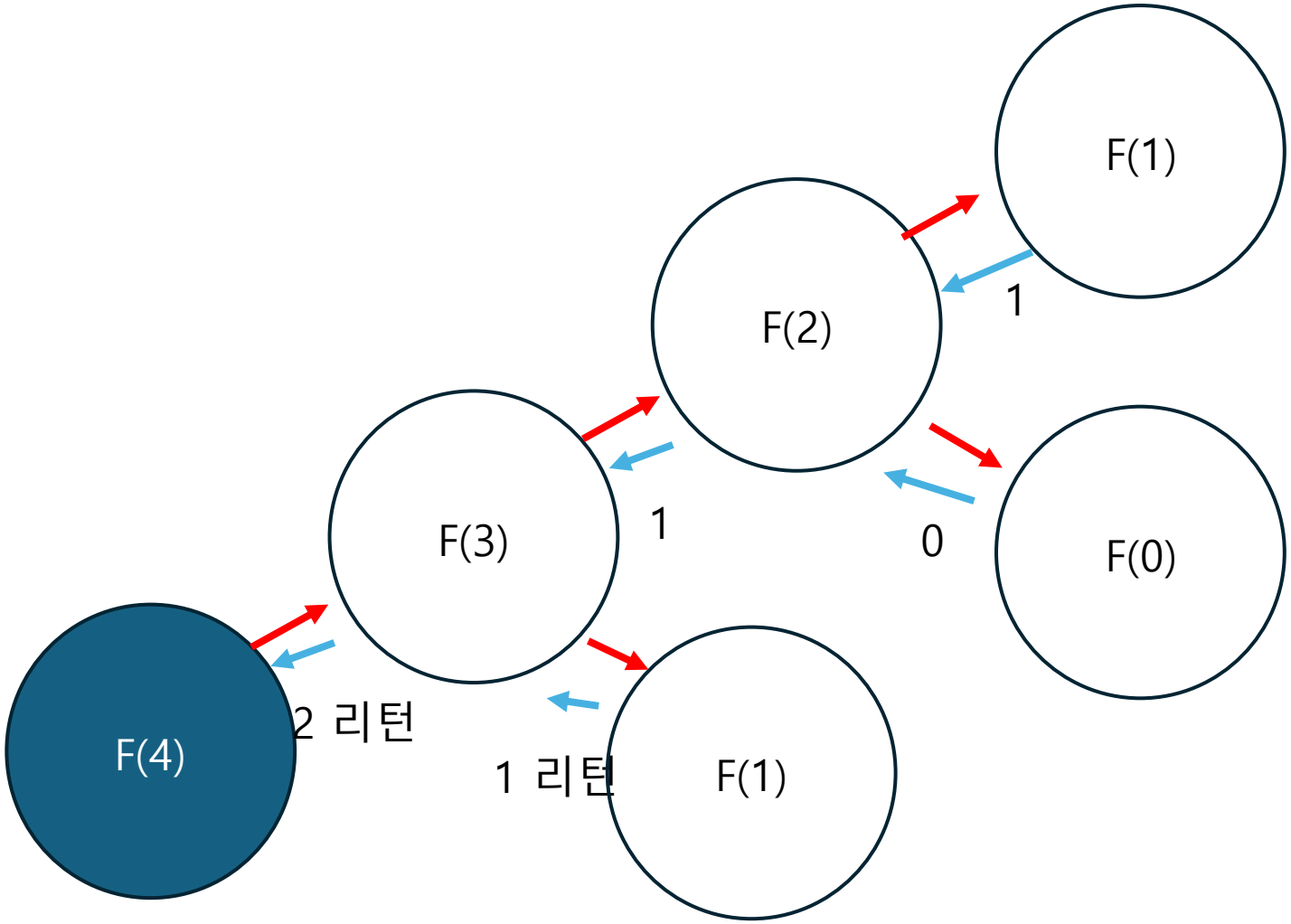
```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```



ㄴ 종료조건을 만족하므로 1을 반환한다. F(3)에는 함수 2개를 호출하는데 F(n-1)에 대응하는 F (2)은 앞에서 이미 계산했고 아직 F(n-2)을 계산하지 않았다는 점이 포인트!

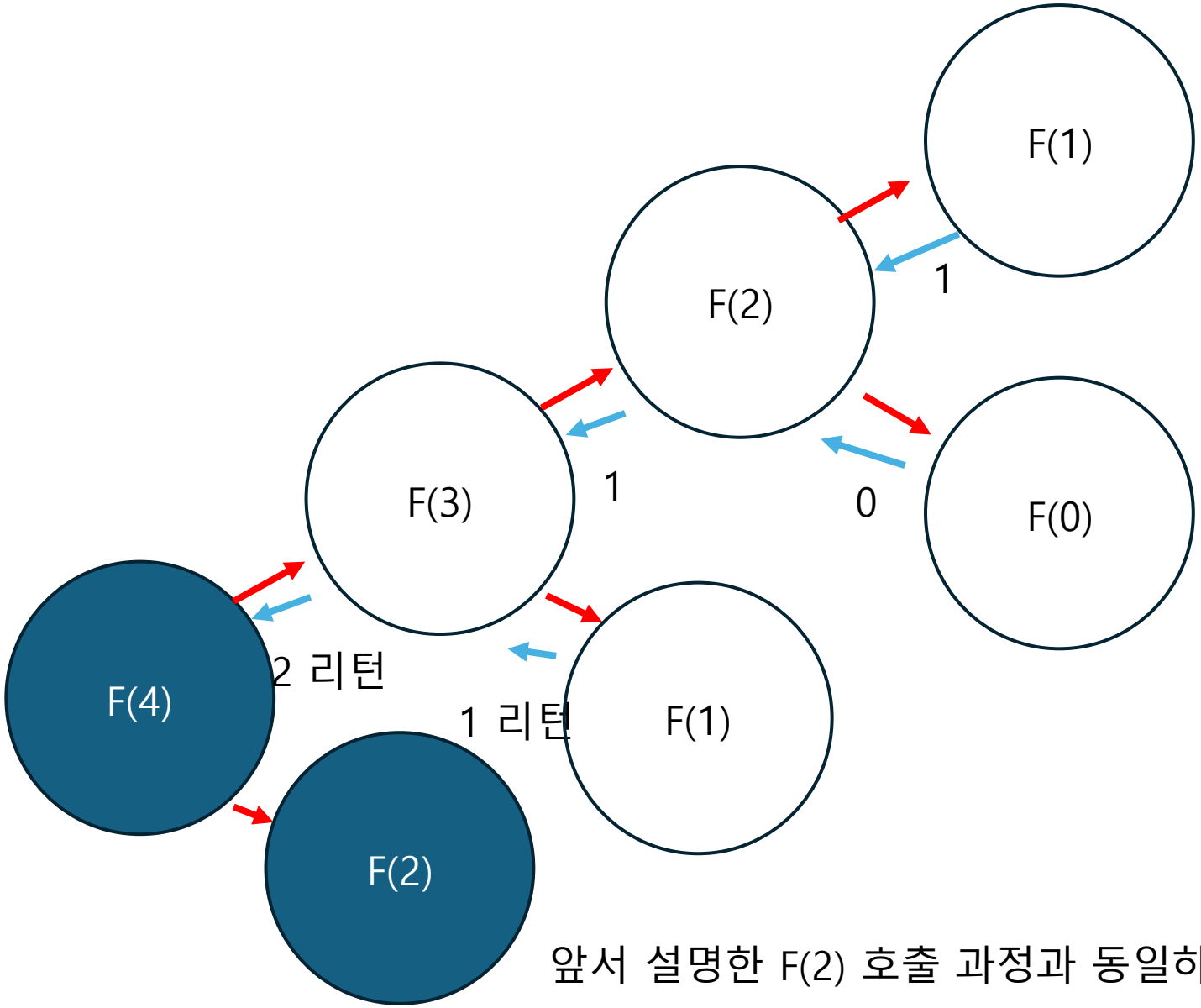
재귀 | 01 재귀의 개념과 원리

```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```



재귀 | 01 재귀의 개념과 원리

```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```

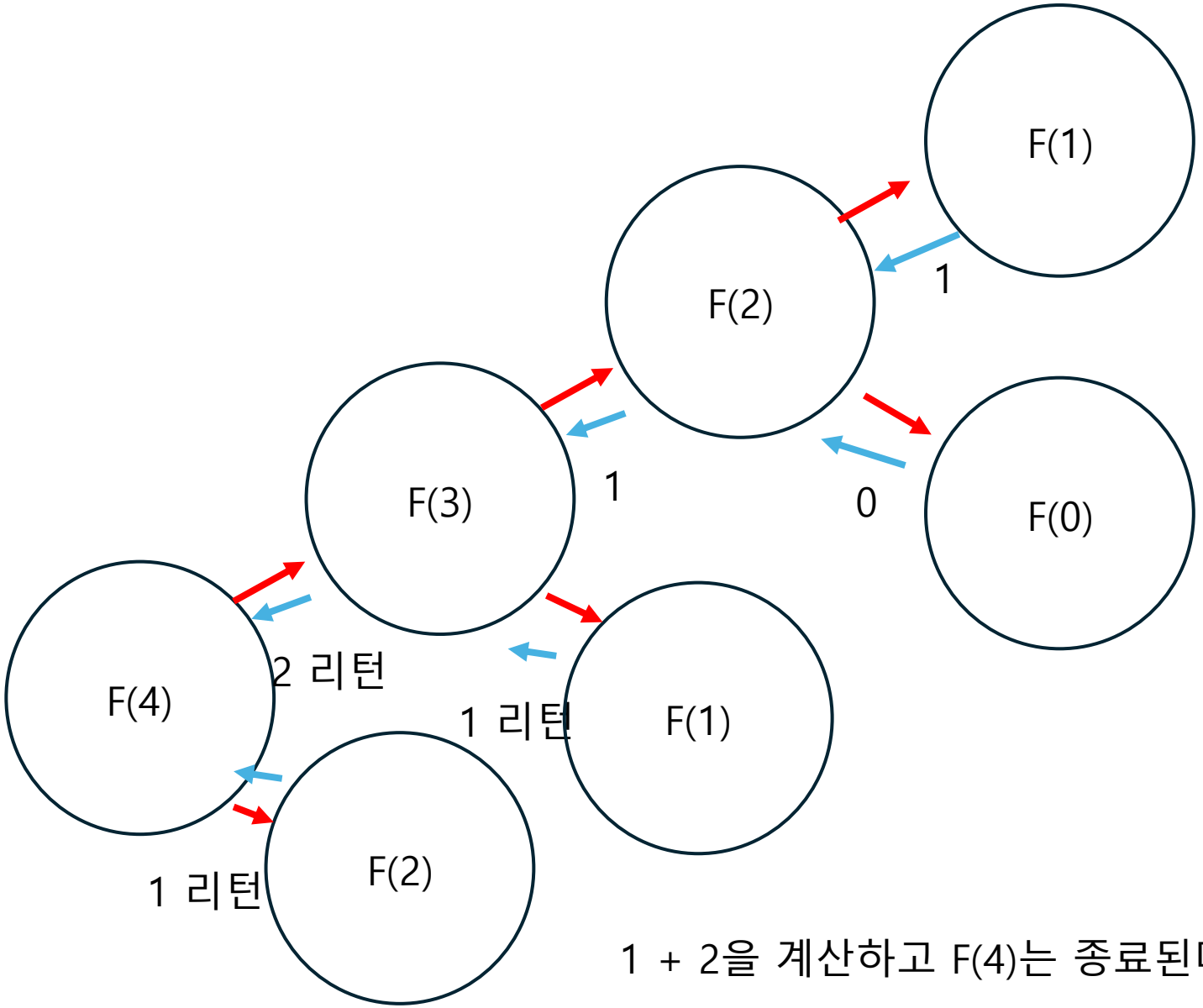


중복되는 계산

앞서 설명한 F(2) 호출 과정과 동일하므로 생략하겠다. 1이 리턴된다.

재귀 | 01 재귀의 개념과 원리

```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```



1 + 2을 계산하고 F(4)는 종료된다.

재귀 | 02 기본적인 재귀 함수 구현

- 팩토리얼 $n! = \begin{cases} 1, & \text{if } n = 0 \text{ or } n = 1 \\ n \times (n-1)!, & \text{if } n \geq 2 \end{cases}$
  - **문제를 작은 문제로 분할하여, 기저 조건에서 종료하도록 설계하면 된다.**

팩토리얼은 F(n) = n * F(n-1)이라는 점화식(단, F(0) = 1)으로 표현되며 수학적 귀납법으로 증명할 수 있다.

<구현>

1. 함수 F(n)은 n! 을 반환하는 함수라고 믿는다(정의한다). // 문제를
2. 그러면 F(n – 1)은 (n – 1)!을 반환해준다. // 작은 문제로 분할하여
3. 가장 작은 입력은 직접 처리해준다. // 기저 조건에서 종료하도록 설계

```
int factorial(int n){
    if(n == 0 or n == 1) return 1;
    return n * factorial(n - 1);
}
```

재귀 | 02 기본적인 재귀 함수 구현

- 피보나치

$$F(n) = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ F(n-1) + F(n-2), & \text{if } n \geq 2 \end{cases}$$

  - 피보나치 수열은 이전 두 개의 숫자를 더해서 다음 숫자를 만드는 수열이다.

    <구현>

    1. 함수 F(n)은 피보나치의 n번째 항을 반환한다고 믿는다(정의한다). // 문제를
    2. 그러면 F(n – 1)은 n – 1번째 항, F(n – 2)은 n – 2번째 항을 반환해준다. // 작은 문제로 분할하여
    3. 가장 작은 입력은 직접 처리해준다. // 기저 조건에서 종료하도록 설계

n 값	함수 호출 횟수
5	15
10	177
20	21,891
30	2,692,537
40	3,541,357,581

```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```

  - 최적화?

    피보나치 수열을 순수한 재귀 함수로 계산하면 중복 계산이 심각하게 많아져 비효율적이다. -> O(2^n)

    Fibo(n)을 호출하면, 두 개의 하위 호출이 발생한다. 연산량이 지수적(2^n)으로 증가하게 된다.

    시간 복잡도, 공간복잡도는 다음 차시에서 다룰 예정.

재귀 | 02 기본적인 재귀 함수 구현

- 최적화? (나중에 다룰 '동적 프로그래밍'의 주된 주제, 복잡도에 관한 내용은 2차시에 학습 예정)

  단순하게, 계산했으면 결과를 저장공간에 저장해두면 된다.

  시간에서 이득을 챙기는 대신, 공간에서 손해를 보는 것이다.

```
int fibo(int n){
    if(n == 0 or n == 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```

시간복잡도 : O(2^n)

```
int result[101]; // -1로 초기화되어있다고 가정
result[0] = 0, result[1] = 1;

int fibo(int n){
    if(result[n] != -1) return result[n];
    return result[n] = fibo(n - 1) + fibo(n - 2);
}
```

시간복잡도 : O(n)

공간복잡도 : O(n)

재귀 | 02 기본적인 재귀 함수 구현

- 피보나치 수열 (반복문)

  피보나치 수열은 반복문으로도 계산 가능하다.

```
10  long long fibonacci(int n) {
11      if (n == 0) return 0;
12      if (n == 1) return 1;
13
14      long long a = 0, b = 1, result = 0;
15      for (int i = 2; i <= n; ++i) {
16          result = a + b;
17          a = b;
18          b = result;
19      }
20      return result;
21  }
```

재귀 | 03 재귀 사용 시 주의할 점

- 재귀가 너무 깊어지면 stack overflow 가 발생할 수 있으므로, Base Case (종료 조건)를 명확히 설정하는 것이 중요하다.
  - 언어별, 시스템별로 허용되는 최대 재귀 깊이를 확인하면 된다.
- 메모이제이션을 활용하여 중복 연산을 방지하고 성능을 최적화할 수 있다. (4차시에 학습할 DP)
- 반복문이 가능한 경우, 반복문이 성능 측면에서 유리할 수 있다.
  - 함수 호출에 따른 비용이 들지 않기 때문이다.

재귀 | 04 예제 : 백준11729번 하노이 탑 이동 순서

- 문제 요약
  - 세 개의 기둥(A, B, C)이 있고, N개의 원판이 A 기둥에 쌓여 있음.
  - 한 번에 하나의 원판만 이동 가능
  - 큰 원판이 작은 원판 위에 올 수 없음.
  - N개의 원판을 A -> C로 최소 횟수로 이동해야한다.
- 입력
  - 첫 번째 장대에 쌓인 원판의 개수 N (1 ≤ N ≤ 20)이 주어진다.
- 출력
  - 첫째 줄에 옮긴 횟수 K를 출력한다.
  - 두 번째 줄부터 수행 과정을 출력한다. 두 번째 줄부터 K개의 줄에 걸쳐 두 정수 A B를 빈칸을 사이에 두고 출력하는데, 이는 A번째 탑의 가장 위에 있는 원판을 B번째 탑의 가장 위로 옮긴다는 뜻이다.

예제 입력 1 복사

```
3
```

예제 출력 1 복사

```
7
1 3
1 2
3 2
1 3
2 1
2 3
1 3
```

재귀 | 04 예제 : 백준11729번 하노이 탑 이동 순서

- N개의 원판을 기둥 A에서 기둥 C로 어떻게 옮길까?

문제(N)를

- 가장 큰 원판부터 기둥 C로 옮기면 됨.

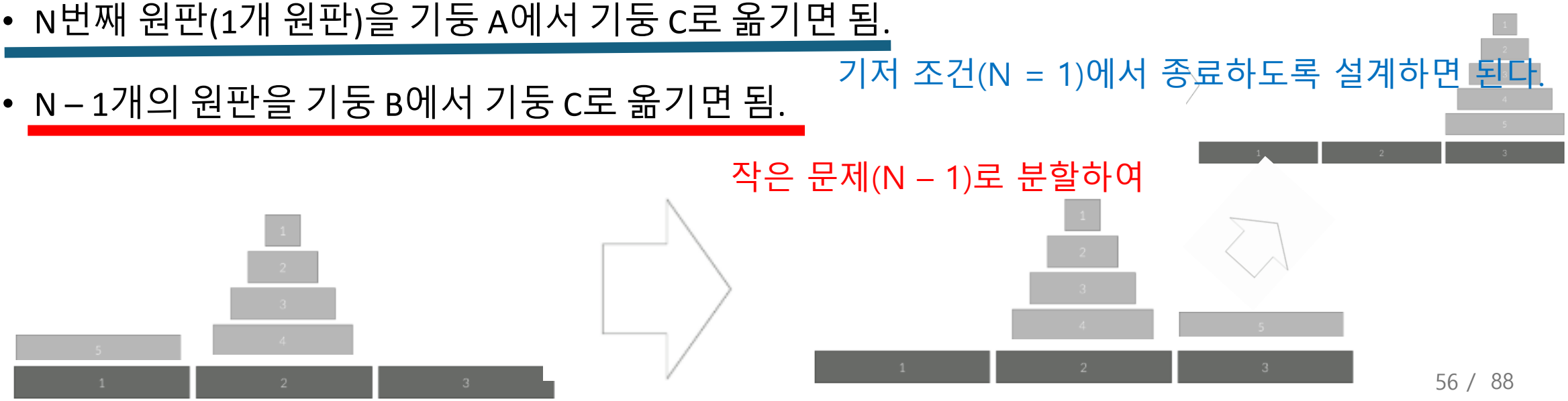
작은 문제(N – 1)로 분할하여

- 가장 큰 원판을 기둥 C로 옮기기 위해 N – 1개의 원판을 기둥 A에서 기둥 B로 옮기면 됨.
- N번째 원판(1개 원판)을 기둥 A에서 기둥 C로 옮기면 됨.

기저 조건(N = 1)에서 종료하도록 설계하면 된다.

- N – 1개의 원판을 기둥 B에서 기둥 C로 옮기면 됨.

작은 문제(N – 1)로 분할하여



재귀 | 04 예제 : 백준11729번 하노이 탑 이동 순서

- f(N, a, b, c) : **N개의 원판을 a에서 c로 옮기는 과정을 구하는 함수이다.**

  f(N – 1, a, c, b): **N – 1개의 원판을 a에서 b로 옮기는 과정을 구하는 함수이다.**

  f(1, a, b, c) : **1개의 원판을 a에서 c로 옮기는 과정을 구하는 함수이다.**

  f(N – 1, b, a, c) : **N – 1개의 원판을 b에서 c로 옮기는 과정을 구하는 함수이다.**

- 가장 작은 입력(N = 1)은 직접 처리한다.
- 종료 조건은 N = 1일때이고 a와 c를 출력하고 종료함.
- 옮긴 횟수는 2^N – 1 이다.

```
#include <bits/stdc++.h>
using namespace std;

void f(int n, int a, int b, int c){
    if(n == 1){
        cout << a << " " << c << "\n";
        return;
    }
    f(n - 1, a, c, b);
    f(1, a, b, c);
    f(n - 1, b, a, c);
    return;
}

int main(){
    int n;
    cin >> n;
    f(n, 1, 2, 3);
}
```

비트마스킹

비트마스킹 | 01 개요

- 비트마스킹

  비트마스킹은 정수의 이진수 표현을 이용하여 **집합을 다루는 기법**이다.

  예를들어, 5 를 이진수로 나타내면 0000 0101 이다.

  1이 있는 위치는 해당 원소가 포함됨을 의미한다.

  0이 있는 위치는 해당 원소가 포함되지 않음을 의미한다.

unsigned long long int가 64비트를 가질 수 있으므로 최대 64개 원소의 표현이 가능하다.

비트마스킹 | 02 비트 연산자

- 비트 연산자

  비트마스킹을 다루기 위해서는 비트 연산자를 알아야 한다.

  비트 연산자는 정수의 이진수 표현을 직접 조작하는 연산자이다.

  연산 속도가 빨라 알고리즘 문제 해결에 요긴하게 쓰인다.

  특히, 비트로 집합을 나타내는 비트마스킹 기법에서 요긴하게 쓰인다.

이외에도 다양한 목적으로 사용한다!

ex) INT64_MAX 대신 ‘(1LL << 63) – 1’ 사용

ex) 어떤 정수에 2를 곱하는 것 대신에 쉬프트 연산자 이용 ’a << 1’

ex) 어떤 정수를 2로 나누는 것 대신에 쉬프트 연산자 이용 ‘a >> 1’

비트마스킹 | 02 비트 연산자

- 비트 연산자 ‘&’ 은 AND 연산을 한다. (이항 연산자)

  전부 참(1)이여야지 참이다. 아닌 경우에는 거짓(0)이다.

  비트마스킹에서 ‘집합에 특정 원소가 존재하는지 여부를 확인’ 할 수 있다.

char : 8 bits	**0**	**1**	**0**	**1**	**0**	**1**	**0**	**1**
	&	&	&	&	&	&	&	&
char : 8 bits	**0**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
	=	=	=	=	=	=	=	=
결과값	**0**	**1**	**0**	**1**	**0**	**1**	**0**	**1**

비트마스킹 | 02 비트 연산자

- 비트 연산자 ‘|’ 은 OR 연산을 한다. (이항 연산자)

  전부 거짓이면 거짓이다. 그렇지 않은 경우 항상 참이다.

  비트마스킹에서 ‘집합에 특정 원소를 추가하는 기능’을 한다.

char : 8 bits	**0**	**1**	**0**	**1**	**0**	**1**	**0**	**1**
	|	|	|	|	|	|	|	|
char : 8 bits	**0**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
	=	=	=	=	=	=	=	=
결과값	**0**	**1**	**1**	**1**	**1**	**1**	**1**	**1**

비트마스킹 | 02 비트 연산자

- 비트 연산자 '^' 은 XOR 연산을 한다. (이항 연산자)

  서로 다를 경우 참이다. 그렇지 않은 경우 거짓이다.

  비트마스킹에서 '집합에 특정 원소를 토글하는 기능'을 한다.

char : 8 bits	**0**	**1**	**0**	**1**	**0**	**1**	**0**	**1**
	^	^	^	^	^	^	^	^
char : 8 bits	**0**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
	=	=	=	=	=	=	=	=
결과값	**0**	**0**	**1**	**0**	**1**	**0**	**1**	**0**

비트마스킹 | 02 비트 연산자

- 비트 연산자 '~' 은 NOT 연산을 한다. (단항 연산자)

  비트를 반전시키는 역할을 한다. (0 -> 1, 1 -> 0)

char : 8 bits	0	0	0	0	0	0	0	0	0

~

결과값	1	1	1	1	1	1	1	1	-1

주의 : '!'와 비슷해보이지만, '!'는 논리연산자이다.

비트마스킹 | 02 비트 연산자

- 비트 연산자 ‘<<’ 은 비트를 n 칸씩 왼쪽으로 민다. (이항 연산자)

  2^n 를 곱한 효과가 나타난다.

  ex) 5 << 1 == 10

char : 8 bits	0	0	0	0	0	1	0	1	5

<< 1

결과값	0	0	0	0	1	0	1	0	10

비트마스킹 | 02 비트 연산자

- 비트 연산자 ‘>> ’ 은 비트를 n 칸씩 오른쪽으로 민다. (이항 연산자)

  2^n 로 나눈 효과가 나타난다.

  ex) 5 >> 1 == 2

char : 8 bits	0	0	0	0	0	1	0	1	5

>> 1

결과값	0	0	0	0	0	0	1	0	2

비트마스킹 | 03 비트 연산 활용

- 특정 비트 체크 : k + 1번째 비트가 1인지 확인, 집합에 원소 유무를 확인

```
if (n & (1 << k)) {
    cout << k << "번째 비트는 1입니다.";
}
```

- 특정 비트 설정 (집합에 원소 추가)

  n |= (1 << (k – 1)); // k번째 비트를 1로 설정 (k >= 1)

- 모든 부분 집합 순회 (조합)

```
#define lld unsigned long long int
for(lld bitset = 0; bitset <= (~0ULL); bitset++){
    ...
}
```

비트마스킹 | 04 예제 : 백준 11723번 집합

- 비어있는 공집합 S가 주어졌을 때, 아래 연산을 수행하는 프로그램을 작성하시오.
  - add x: S에 x를 추가한다. (1 ≤ x ≤ 20) S에 x가 이미 있는 경우에는 연산을 무시한다.
  - remove x: S에서 x를 제거한다. (1 ≤ x ≤ 20) S에 x가 없는 경우에는 연산을 무시한다.
  - check x: S에 x가 있으면 1을, 없으면 0을 출력한다. (1 ≤ x ≤ 20)
  - toggle x: S에 x가 있으면 x를 제거하고, 없으면 x를 추가한다. (1 ≤ x ≤ 20)
  - all: S를 {1, 2, ..., 20} 으로 바꾼다.
  - empty: S를 공집합으로 바꾼다.

입력

첫째 줄에 수행해야 하는 연산의 수 M (1 ≤ M ≤ 3,000,000)이 주어진다.

둘째 줄부터 M개의 줄에 수행해야 하는 연산이 한 줄에 하나씩 주어진다.

출력

check 연산이 주어질때마다, 결과를 출력한다.

예제 입력 1 복사

```
26
add 1
add 2
check 1
check 2
check 3
remove 2
check 1
check 2
toggle 3
check 1
check 2
check 3
check 4
all
check 10
check 20
toggle 10
remove 20
check 10
check 20
empty
check 1
toggle 1
check 1
toggle 1
check 1
```

예제 출력 1 복사

```
1
1
0
1
0
1
0
1
0
1
1
0
0
0
1
0
```

비트마스킹 | 04 예제 : 백준 11723번 집합

- 풀이

Line 9 ~ 10

입출력을 빠르게 하기 위한 코드로, 쓰지 않으면 시간초과이다.

Line 14, 17, 20, 23

x를 1 감소시켜주는 이유는 1 자체로 첫번째 원소를 나타내기 때문이다.

Line 13 ~ 15

add 연산을 구현했다. 집합에 원소를 추가하는 연산은 OR 연산으로 가능하다.

Line 16 ~ 18

remove 연산을 구현했다. ~(1 << x)은 집합 S의 x번째 원소를 제외하고 전부 1인 상태이다.
이를 기존 집합인 S와 AND 연산을 해주면 집합 S의 x번째 원소를 제거할 수 있다.

Line 19 ~ 21

check 연산을 구현했다. 집합에 특정 원소가 있는지 확인하는 연산은 AND 연산으로 가능하다.

Line 22 ~ 24

toggle 연산을 구현했다.

Line 25 ~ 28

all 연산은 엄밀하진 않지만 INT32_MAX로 표현할 수 있고, empty 연산은 0으로 표현할 수 있다.

```
1   #include <bits/stdc++.h>
2   using namespace std;
3
4   int S = 0, M, x;
5   string cmd;
6
7   int main(){
8       cin >> M;
9       ios::sync_with_stdio(false);
10      cin.tie(nullptr);
11      while(M--){
12          cin >> cmd;
13          if(cmd == "add"){
14              cin >> x; x--;
15              S |= (1 << x);
16          }else if(cmd == "remove"){
17              cin >> x; x--;
18              S = S & ~(1 << x);
19          }else if(cmd == "check"){
20              cin >> x; x--;
21              cout << (S & (1 << x) ? 1 : 0) << "\n";
22          }else if(cmd == "toggle"){
23              cin >> x; x--;
24              S ^= (1 << x);
25          }else if(cmd == "all"){
26              S = INT32_MAX;
27          }else{
28              S = 0;
29          }
30      }
31  }
```

비트마스킹 | 05 쓰이는 곳

- 비트마스킹은 어디에 쓰일까?
  - 상태 관리

    아까 예제처럼 관리해야 하는 수가 최대 64개 이하(unsigned long long int)일 때 유용하다.
  - 부분 집합 ex) 백준 15661번 링크와 스타트

    N개의 원소가 있을 때, 2^N개의 부분 집합을 비트마스킹을 이용하여 빠르게 생성할 수 있다. (N <= 64)

    공집합을 나타내는 0부터, 전체 집합을 나타내는 (1ull << (N – 1)) 까지를 반복문으로 순회하면 모든 부분집합을 생성할 수 있다.
  - 최적화 ex) 백준 2098번 외판원 순회

    나중에 배울 동적 프로그래밍과 엮여 다양하게 활용된다.

백트래킹

백트래킹 | 01 기본 개념

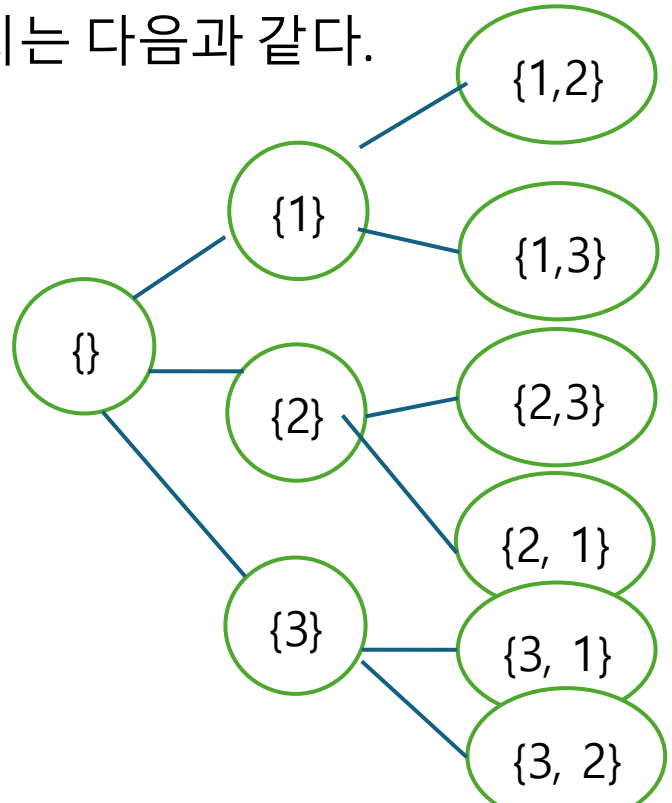
- 백트래킹이란?
  - 모든 가능한 경우를 고려하면서도 불필요한 탐색을 줄이는 탐색 기법이다.
  - 즉, 백트래킹은 탐색 과정에서 해당 경로가 유망하지 않다면 더 이상 진행하지 않고 되돌아가는 기법을 활용하여 탐색 공간을 줄이고 연산량을 감소시킨다.

백트래킹 | 01 기본 개념

- 기본 원리
  1. 모든 경우를 고려하며 탐색을 진행한다. (= 브루트 포스, 완전 탐색)
  2. 탐색을 진행하던 중, 현재 상태가 해결책이 될 가능성이 없는 경우(불가능한 경로)라면 더 깊이 탐색하지 않고 가지치기(Pruning)를 수행하여 탐색을 중단하고 이전 단계로 되돌아간다. (백트래킹의 특징)
  3. 위 과정을 반복하여 가능한 모든 해를 찾거나 최적의 해를 구한다.

백트래킹 | 01 기본 개념

- 상태 공간 트리(State Space Tree)
  - 문제를 해결하기 위한 탐색 공간을 트리 형태로 표현한 구조이다.
  - 예) 백준 15649번 N과 M (1) : 1부터 N까지 자연수 중에서 중복 없이 M개를 고른 수열 출력, N = 3, M = 2일 때의 상태 공간 트리는 다음과 같다.



백트래킹 | 01 기본 개념

- 가지치기(Pruning)의 개념과 필요성
  - 가지치기는 탐색 알고리즘에서 불필요한 탐색 경로를 조기에 차단하여 탐색 공간을 줄이는 기법이다.
  - 모든 경우를 탐색해야 하는 완전 탐색(Brute Force) 방식은 비효율적이므로, 불필요한 경로는 미리 배제하여 탐색 속도를 향상시키는 것이 가지치기의 핵심 원리이다.

백트래킹 | 02 구현 방법

- 백트래킹 기본 구조

```
void backtrack(현재 상태) {
    if (정답 조건을 만족하면) {
        정답 처리;
        return;
    }

    for (모든 가능한 선택) {
        if (유효한 선택인지 검사) {  // 가지치기 (Pruning)
            선택 수행;
            backtrack(다음 상태);  // 재귀 호출 (다음 단계 탐색)
            선택 취소 (이전 상태로 되돌리기);  // 백트래킹
        }
    }
}
```

핵심 개념

1. 현재 상태를 기준으로 가능한 모든 선택을 시도
2. 유효하지 않은 경우 즉시 배제하여 탐색을 줄임
3. 유효한 경우에만 재귀 호출(다음 상태로 이동)
4. 탐색이 끝나면 선택을 취소하고 원래 상태로 복구

백트래킹 | 03 예제 (1) : 백준 15649번 N과 M (1)

- 문제 요약
  - 1부터 N까지의 자연수 중에서 중복 없이 M개를 고른 수열을 모두 출력하는 문제이다.
  - 수열은 사전 순으로 출력해야 한다.
  - 예) N = 3, M = 2
- 입력
  - 첫째 줄에 자연수 N과 M이 주어진다. (1 ≤ M ≤ N ≤ 8)
- 출력
  - 한 줄에 하나씩 문제의 조건을 만족하는 수열을 출력한다.
  - 수열은 사전 순으로 증가하는 순서로 출력해야 한다.

예제 입력 2 복사

```
4 2
```

예제 출력 2 복사

```
1 2
1 3
1 4
2 1
2 3
2 4
3 1
3 2
3 4
4 1
4 2
4 3
```

백트래킹 | 03 예제 (1) : 백준 15649번 N과 M (1)

- **func(m) : 지금까지 선택한 수는 m개이다.**
  - m = M 이면 출력하고 종료 (정답 처리)
    - m : 지금까지 선택한 수의 개수
    - M : 선택 해야하는 수의 개수
  - v는 지금까지 선택한 수를 저장하는 벡터이다.
  - vis[i]가 참이면, 숫자 i는 쓰였다는 의미이다. 거짓이라면, 안 쓰였다는 의미다.

백트래킹의 기본 구조

```
void backtrack(현재 상태) {
    if (정답 조건을 만족하면) {
        정답 처리;
        return;
    }

    for (모든 가능한 선택) {
        if (유효한 선택인지 검사) {  // 가지치기 (Pruning)
            선택 수행;
            backtrack(다음 상태);  // 재귀 호출 (다음 단계 탐색)
            선택 취소 (이전 상태로 되돌리기);  // 백트래킹
        }
    }
}
```

```
#include bits/stdc++.h
using namespace std;

int N, M;
bool vis[10];
vector<int> v;

void func(int m){
  if(v.size() == M){
    for(auto &x : v) cout << x << " ";
    cout << "\n";
    return;
  }

  for(int i = 1; i <= N; i++){
    if(vis[i] == false){
      vis[i] = true;
      v.push_back(i);
      func(m + 1);
      v.pop_back();
      vis[i] = false;
    }
  }
  return;
}

int main(){
  cin >> N >> M;
  func(0);
  return 0;
}
```

지금까지 선택한 수는 0개이다

백트래킹 | 03 예제 (1) : 백준 15649번 N과 M (1)

- 모든 가능한 선택을 시도
- 가지치기 : 유효한 선택인지 검사.
- 선택 수행
- 다음 단계 탐색
- 선택 취소

백트래킹의 기본 구조

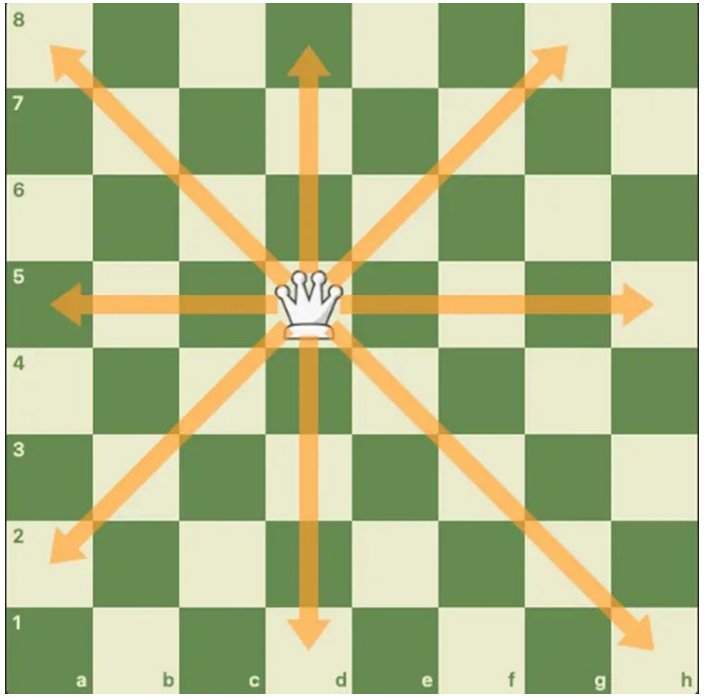
```
void backtrack(현재 상태) {
    if (정답 조건을 만족하면) {
        정답 처리;
        return;
    }

    for (모든 가능한 선택) {
        if (유효한 선택인지 검사) {  // 가지치기 (Pruning)
            선택 수행;
            backtrack(다음 상태);  // 재귀 호출 (다음 단계 탐색)
            선택 취소 (이전 상태로 되돌리기);  // 백트래킹
        }
    }
}
```

```
1  #include bits/stdc++.h
2  using namespace std;
3
4  int N, M;
5  bool vis[10];
6  vector<int> v;
7
8  void func(int m){
9    if(v.size() == M){
10     for(auto &x : v) cout << x << " ";
11     cout << "\n";
12     return;
13   }
14
15   for(int i = 1; i <= N; i++){
16     if(vis[i] == false){
17       vis[i] = true;
18       v.push_back(i);
19       func(m + 1);
20       v.pop_back();
21       vis[i] = false;
22     }
23   }
24   return;
25 }
26
27 int main(){
28   cin >> N >> M;
29   func(0);
30   return 0;
31 }
```

백트래킹 | 03 예제 (2) : 백준 9663번 N-Queen

- 문제 요약
  - N-Queen 문제는 크기가 N × N인 체스판 위에 퀸 N개를 서로 공격할 수 없게 놓는 문제이다.
  - N이 주어졌을 때, 퀸을 놓는 방법의 수를 구하자.

참고) 퀸은 같은 행, 열, 대각선 상에 있으면 공격 가능하다.



예제 입력 1 복사

```
8
```

예제 출력 1 복사

```
92
```

백트래킹 | 03 예제 (2) : 백준 9663번 N-Queen

- 퀸은 같은 행, 열, 대각선 상에 있으면 공격 가능하다.
  1. 0번째 행부터 N개의 퀸을 배치
  2. 같은 열과 2개의 대각선에 다른 퀸이 있는지 검사
  3. 조건을 만족하면 다음 행으로 이동
  4. N개의 퀸을 모두 배치하면 경우의 수 증가

     N번째 행에 도달하면 N개의 퀸을 모두 배치했다는 것이다. (0-indexed)
- 같은 열, 대각선에 다른 퀸이 있는지 검사하는 방법?
  - bool col[N]; 라는 배열을 정의하자.
  - col[a] = true이면, a열에 퀸을 두지 못한다는 의미이다.

백트래킹 | 03 예제 (2) : 백준 9663번 N-Queen

- 같은 열과 2개의 대각선에 다른 퀸이 있는지 검사하는 방법
  - bool col[N]; // col[a] = true이면, 해당 열에 퀸을 두지 못한다.
  - row + col이 같은 값이면, '/' 대각선에 위치한다.
  - row – col이 같은 값이면, '\' 대각선에 위치한다. (음수 값 피하기 위해 같은 수를 더해주면 됨)

(row - col 값, \ 대각선)

plaintext

```
(0,0)  0   (0,1) -1   (0,2) -2   (0,3) -3
(1,0)  1   (1,1)  0   (1,2) -1   (1,3) -2
(2,0)  2   (2,1)  1   (2,2)  0   (2,3) -1
(3,0)  3   (3,1)  2   (3,2)  1   (3,3)  0
```

→ 같은 숫자가 같은 \ 대각선

(row + col 값, / 대각선)

plaintext

```
(0,0)  0   (0,1)  1   (0,2)  2   (0,3)  3
(1,0)  1   (1,1)  2   (1,2)  3   (1,3)  4
(2,0)  2   (2,1)  3   (2,2)  4   (2,3)  5
(3,0)  3   (3,1)  4   (3,2)  5   (3,3)  6
```

→ 같은 숫자가 같은 / 대각선

백트래킹 | 03 예제 (2) : 백준 9663번 N-Queen

```
void backtrack(현재 상태) {
    if (정답 조건을 만족하면) {
        정답 처리;
        return;
    }

    for (모든 가능한 선택) {
        if (유효한 선택인지 검사) {
            선택 수행;
            backtrack(다음 상태);  // 재귀 호출 (다음 단계 탐색)
            선택 취소 (이전 상태로 되돌리기);  // 백트래킹
        }
    }
}
```

void func(int row) : 현재 row번째 행까지 퀸을 놓은 상태

row == N이면 모든 퀸을 놓은 상태이므로 정답처리

모든 열에 퀸을 놓아보면서 유효한 선택인지 검사

놓으려는 위치에 열과 2개의 대각선에 퀸이 없으면 됨.

```
15        if(!col[c] && !diag1[row - c + N] && !diag2[row + c]){
```

백트래킹 | 03 예제 (2) : 백준 9663번 N-Queen

- Line. 9 ~ 12

  N개의 퀸을 전부 놓았으면 경우의 수 증가 시키고 종료

- Line. 14 ~ 20

  15 : 유효성 검사(가지치기)

  16 : 선택 수행

  17 : 다음 단계 탐색

  18 : 선택 취소

row를 현재 row - 1번째 행까지 퀸을 놓은 상태라고 생각하면 된다.

```
1   #include <bits/stdc++.h>
2   using namespace std;
3
4   int N, res;
5   bool col[15], diag1[30], diag2[30];
6
7
8   void func(int row){
9       if(row == N){
10          res++;
11          return;
12      }
13
14      for(int c = 0; c < N; c++){
15          if(!col[c] && !diag1[row - c + N] && !diag2[row + c]){
16              col[c] = diag1[row - c + N] = diag2[row + c] = true;
17              func(row + 1);
18              col[c] = diag1[row - c + N] = diag2[row + c] = false;
19          }
20      }
21  }
22
23  int main(){
24      cin >> N;
25      func(0);
26      cout << res << "\n";
27      return 0;
28  }
```

백트래킹 | 04 가지치기 유형

1. 유망성 검사
   - 현재 상태에서 정답이 될 가능성이 있는지 검사하여, 가능성이 없으면 더 이상 탐색하지 않음.
   - 예) 미로를 탈출할 때 벽이 막힌 공간을 굳이 탈출할 필요 없음.
2. 하한/상한 제한
   - 가능한 해의 범위를 설정하여, 해당 범위를 벗어나면 탐색하지 않음.
   - 예) N개의 수가 담긴 배열에서 M개를 선택해서 모두 더한 것들 중에서 K보다 작으면서 가장 큰 조합을 고르기. 당연히 M개를 선택하는 과정에서 K보다 커지면 탐색을 중지해야함.
3. 최적해를 위한 가지치기
   - 현재 상태에서 더 진행해도 지금까지 계산해낸 최적해보다 나빠지는 경우 탐색 종료
   - 예) 경로 진행 중에 지금까지 구한 최적해보다 비용이 커지면 탐색할 필요가 없으므로 종료.

결국은, 불필요한 경로를 제거하는 전략을 고민하는 게 가지치기의 핵심

문제 | 재귀, 백트래킹 문제집 위주로 푸시면 됩니다!

10870 피보나치 수 5

15649, 15650, 15651, 15652번 N과 M

1182 부분수열의 합

2580 스도쿠

9663 N-Queen

11729 하노이 탑 이동 순서

14888 연산자 끼워넣기

14889 스타트와 링크

추천 문제집

https://www.acmicpc.net/workbook/view/7314

https://www.acmicpc.net/workbook/view/7315

오늘 스터디는 여기서 마무리하겠습니다.

백준 많이 푸세요! 수고하셨습니다!